

LANGCHAIN İLE BELGELER ÜZERİNDE SORU-CEVAP (RAG)

Harrison Chase Ders Çalışma Raporu

1. Giriş ve Temel Problem: LLM'lerin Bağlam Sınırı

Büyük Dil Modelleri (LLM'ler) ile geliştirilen en popüler ve kritik uygulamalardan biri, mevcut belgeler (PDF'ler, CSV'ler, web sayfaları veya kurumsal intranet dokümanları) üzerinde soru-cevap yapabilen akıllı sistemlerdir. Bu sistemler, LLM'lerin önceden eğitilmediği tescilli veya özel veriler üzerinde akıl yürütmesini sağlayarak onları son derece esnek ve uyarlanabilir hale getirir. Ancak bu noktada büyük bir teknik engel karşımıza çıkar: **LLM'lerin bağlam pencereleri sınırlıdır** ve tek bir seferde yalnızca birkaç bin kelimeyi inceleyebilirler. Binlerce sayfadan oluşan doküman koleksiyonlarını doğrudan modele girdi olarak vermek bu limitler nedeniyle imkansızdır. Bu kısıtı aşmak amacıyla, LangChain framework'ü bünyesinde **Embeddings (Metin Temsilleri)** ve **Vector Stores (Vektör Depoları)** gibi bileşenler kullanılarak bilgi geri çağırımı (retrieval) gerçekleştirilir.

Günlük Hayattan Benzetme: Büyük bir kütüphanede yüzlerce kitap arasında belirli bir konuya dair araştırma yaptığınızı hayal edin. Kütüphanedeki tüm kitapları aynı anda elinize alıp okumanız fiziksel olarak imkansızdır (LLM'lerin bağlam sınırı). Bunun yerine kütüphane kataloğundan (vektör veritabanı) ilgili anahtar kelimeleri aratır, sadece ilgili 3-4 kitabı (en benzer doküman parçaları) masanıza getirir ve yalnızca o bölümleri okuyarak sorunuza cevap bulursunuz.

2. Embeddings (Metin Temsilleri) ve Vektör Uzayı

Metin temsil modelleri (embeddings), herhangi bir metin parçasını alıp onun **anlamsal (semantik) içeriğini** temsil eden sayısal vektörlere (sayı listeleri) dönüştürür. Anlamsal olarak benzer içeriğe sahip metin parçaları, vektör uzayında birbirine çok yakın konumlarda yer alırken, farklı konuları işleyen metinler uzak konumlarda yer alır. Bu sayısal vektör uzayı karşılaştırması, LLM'e hangi doküman parçalarının gönderilmesi gerektiğini belirlemek için kullanılır.

Günlük Hayattan Benzetme: Elimizde üç cümle olduğunu düşünelim:

- 'Benim köpeğim kemik çiğnemeyi çok sever.' (Evcil Hayvan konusu)
- 'Kediler genellikle kendi hallerinde olmayı tercih eder.' (Evcil Hayvan konusu)
- 'Yeni aldığımız arabanın yakıt tüketimi oldukça düşük.' (Otomobil konusu)

Bu cümlelerin embeddings vektörlerini çıkartıp uzayda karşılaştırdığımızda; kedi ve köpek hakkındaki ilk iki cümlenin vektörleri anlamsal yakınlıktan dolayı birbirine çok yakın çıkacaktır. Otomobilden bahseden üçüncü cümle ise bu iki vektörden tamamen uzak ve bağımsız bir noktada konumlanacaktır.

LangChain'de OpenAI'nın embeddings sınıfları kullanılarak bir metin sorgusu vektöre dönüştürüldüğünde, örneğin 'Hi, my name is Harrison' cümlesi için **1000'den fazla nümerik öğeden oluşan** devasa bir vektör elde

edilir. Bu yüksek boyutlu vektör, metnin dünyadaki diğer tüm kavramlarla olan anlamsal ilişkisini hassas bir şekilde matematiksel olarak temsil eder.

3. Vektör Veritabanları ve Parçalama (Chunking)

Büyük boyutlu dokümanlar sisteme yüklendiğinde, öncelikle **küçük metin parçalarına (chunks)** bölünürler. Her bir parçanın ayrı ayrı embedding vektörleri çıkarılır ve bu vektörler ile orijinal metin parçaları bir **vektör veritabanında (Vector Database)** indekslenerek saklanır. Çalışma zamanında (runtime) bir kullanıcı sorgusu geldiğinde sistem şu adımları izler:

- Kullanıcı sorgusunun embedding vektörü oluşturulur.
- Bu vektör, veritabanındaki tüm doküman vektörleriyle matematiksel olarak karşılaştırılır.
- Sorguya anlamsal olarak en benzer n adet metin parçası (doküman) seçilir.
- Seçilen bu parçalar, orijinal soru ile birlikte LLM'e gönderilerek nihai doğal dil cevabı üretilir.

4. LangChain ile Hızlı RAG Uygulaması (Tek Satır Kod)

LangChain, hızlı prototipleme için tüm bu karmaşık arka plan süreçlerini (yükleme, vektörleştirme ve indeksleme) tek satırda halleden yüksek seviyeli yardımcı sınıflar sunar. Aşağıdaki kod örneğinde, outdoor kıyafetleri içeren bir CSV dosyasından VectorStoreIndexCreator kullanılarak nasıl hızlıca soru-cevap yapılabileceği gösterilmiştir:

```
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI
from langchain.document_loaders import CSVLoader
from langchain.vectorstores import DocArrayInMemorySearch
from langchain.indexes import VectorStoreIndexCreator

# 1. Belge yükleyiciyi tanımla (Proprietary CSV Verisi)
loader = CSVLoader(file_path="outdoor_clothing.csv")

# 2. Tek satırda bellek-içi vektör deposu ve indeks oluştur
index = VectorStoreIndexCreator(
    vectorstore_cls=DocArrayInMemorySearch
).from_loaders([loader])

# 3. İndeks üzerinden sorgu yap ve yanıt al
query = "Please suggest a shirt with sunblocking"
response = index.query(query)
```

5. Adım Adım Detaylı RAG Yapısı (Arka Plandaki İşleyiş)

Tek satırlık sihirli kod başlangıç için harikadır; fakat parametreleri özelleştirmek ve arka plandaki akışı kontrol etmek için süreci manuel adımlara bölmek gerekir. İşte aynı sistemin detaylı ve kontrol edilebilir kurulumu:

```
from langchain.document_loaders import CSVLoader
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import DocArrayInMemorySearch
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# Adım 1: Belge Yükleme
loader = CSVLoader(file_path="outdoor_clothing.csv")
docs = loader.load()

# Adım 2: Embeddings Sınıfını Hazırlama ve Sorgu Örneği
embeddings = OpenAIEmbeddings()
sample_embed = embeddings.embed_query("Hi, my name is Harrison")
# Sonuç: >1000 elemanlı sayı dizisi (örn: [-0.012, 0.023, ...])

# Adım 3: Vektör Deposunu Belgelerle Doldurma
db = DocArrayInMemorySearch.from_documents(docs, embeddings)

# Adım 4: Benzerlik Araması (Doküman seviyesinde arama)
query = "Please suggest a shirt with sunblocking"
docs_sub = db.similarity_search(query) # En benzer 4 belgeyi çeker

# Adım 5: RetrievalQA Zinciri (LLM ve Retriever Entegrasyonu)
retriever = db.as_retriever()
llm = ChatOpenAI(temperature=0.0)

qa_stuff = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    verbose=True
)
response = qa_stuff.run(query)
```

6. Geri Çağırılan Belgeleri Birleştirme Yöntemleri (Chain Types)

Veritabanından geri çağırılan ilgili doküman parçalarının dil modeline (LLM) nasıl aktarılacağı ve nasıl birleştirileceği projenin maliyeti, hızı ve doğruluğu açısından kritiktir. LangChain bu birleştirme işlemi için 4 temel yöntem sunar. En yaygın kullanılan varsayılan yöntem **'stuff'** yöntemidir, fakat büyük ölçekli belgeler için alternatif zincir türleri tercih edilmelidir:

Yöntem	Çalışma Prensibi	Avantajları	Dezavantajları
Stuff (Doldurma)	Geri çağırılan tüm doküman parçalarını doğrudan tek bir promptte 'doldurur' ve LLM'e gönderir.	- Çok basit ve hızlıdır. - Tek bir LLM çağırısı gerektirdiği için oldukça ucuzdur. - Bağlam bütünlüğü korunur.	LLM'in bağlam penceresi (token sınırı) aşıldığında sistem hata verir.
Map Reduce	Her bir parçayı soru ile birlikte bağımsız olarak LLM'e gönderip alt yanıtlar alır. Sonra bu yanıtları tek bir LLM çağırısıyla nihai özete dönüştürür.	- Sınırsız boyuttaki belgelerle çalışabilir. - Bağımsız çağrılar paralel yapılabildiği için görece hızlıdır.	- Çok sayıda LLM çağırısı yaptığı için maliyetlidir. - Parçaları bağımsız işlediği için anlamsal ilişkiler kaybolabilir.
Refine (Geliştirme)	Dokümanlar üzerinde sırayla döner. İlk dokümandan aldığı yanıtı, bir sonraki dokümanla birleştirerek cevabı sürekli günceller.	Bilgileri ardışık olarak harmanladığı için son derece detaylı ve derinlemesine yanıtlar üretir.	- Çağrılar birbirine bağımlı olduğu için paralelleştirilemez. - İşlem süresi uzundur ve maliyetlidir.
Map Rerank (Sıralama)	Her doküman için bağımsız bir LLM çağırısı yapar. LLM'den hem cevabı hem de cevabın soruya uygunluk skorunu ister. En yüksek skora sahip cevabı seçer.	- Tekil ve net cevaplar içeren dokümanlarda çok başarılıdır. - Bağımsız çağrılar paralel çalıştırılabilir.	- LLM'in skorlama yeteneğine bağımlıdır. - Prompt detaylandırılması zordur. - Çok sayıda çağrı nedeniyle maliyetlidir.

Önemli Çıkarımlar ve Mimari Kararlar

- **Varsayılan Tercih:** Küçük ve orta ölçekli belgelerle çalışırken (veya similarity search ile sadece en benzer birkaç parça çekildiğinde) en mantıklı ve ekonomik yöntem **Stuff** yöntemidir.
- **Büyük Doküman Özeti:** Çok büyük belgeleri recursive (öz yinelemeli) olarak özetlemek istiyorsanız, **Map Reduce** en ideal alternatiftir.
- **Akıllı ve Detaylı Analiz:** Yanıtın her sayfadaki bilgiyle adım adım olgunlaşmasını istiyorsanız **Refine** mimarisini tercih etmelisiniz.

Rapor Sonu - Bu belge, 'LangChain for LLM Application Development' eğitim serisindeki orijinal kaynaklar ve kod pratikleri temel alınarak hazırlanmıştır.